REST

Creating the programmable web

Aims for today

- Understand what REST is and what it means
- Why it’s important
- Basics of using and creating RESTful applications

What problem are we trying to solve?

- The functions in your controller classes represent an interface
- When client-side code makes a call to the server, it is using that server-side interface
- As we put more and more code into the client, the client-side part of the application becomes more complex
- We can help reduce that complexity by creating simple and easy-to-use server-side interfaces that result in simpler client-side code
- The overall ambition of REST is to create an “architectural style” that creates simple, easy-to-use server-side interfaces based on CRUD principles

REST

- REST = REpresentational State Transfer
- An architectural style for the web - i.e., a set of characteristics that defines a set of things
- Benefits - using REST, web applications become components - that can be bolted/fitted together easily to create new applications
- Enables us to build some interesting applications
  - Makes it easier to compose things together

Why REST?

- REST enables developers to interfaces that are easy to understand and use
- Decouples the consumer of information (the client) from the producer of information (the server)
- We can build services in layers, using information from one REST interface to build another

Examples of REST APIs

- Monzo (internet bank) https://docs.monzo.com (needs authentication token)
- NASA https://api.nasa.gov
- API-football https://rapidapi.com/api-sports/api/api-football (requires sign-up to rapidapi)
- Chuck Norris API https://api.chucknorris.io
- https://github.com/public-api-lists/public-api-lists

Another example – the Star Wars API

- https://www.swapi.info/api
- https://www.swapi.info/api/films – all the Star Wars films in JSON format
- https://swapi.info/api/films/1 - JSON data on Star Wars Episode 4, “A New Hope”
- https://swapi.info/api/people/1 - JSON data on Luke Skywalker
- See https://github.com/SivaramPg/swapi.info for the website code (Node.js + typescript)

Resources and representations

- REST makes a distinction between a resource and its representation

This is a dummy URL, btw

- http://www.kitchen.com/utensil/cups/enamelcup
  - Is a resource
  - If you request this resource, you get a **representation** of the resource
  - There can be different representations

Example Resource

http://mapit.mysociety.org/postcode/W1W6UW

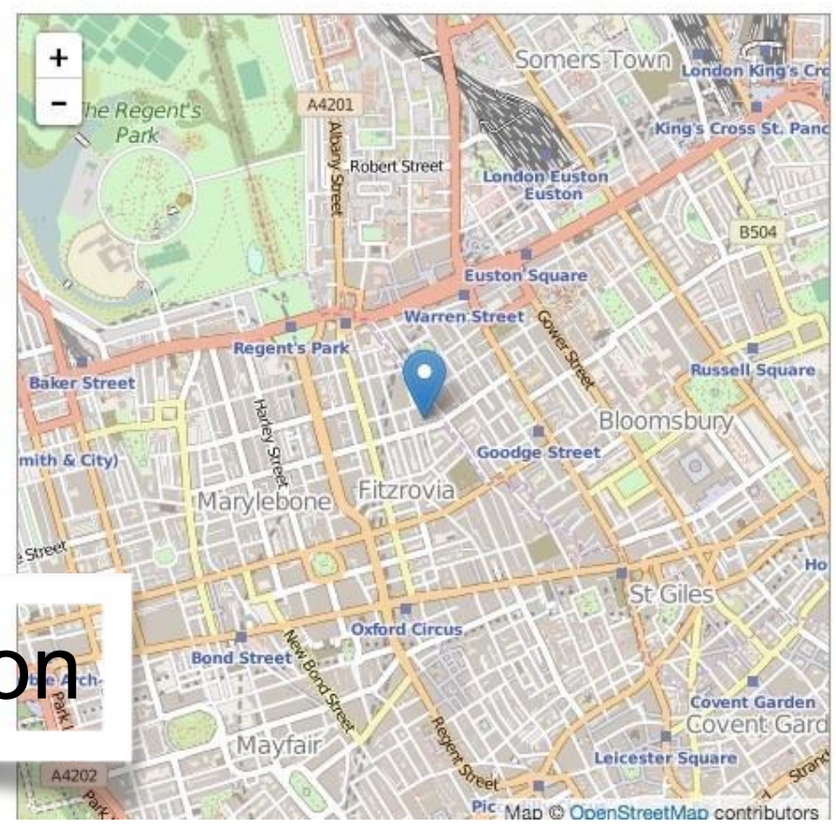
URL names this resource

Representations


```
{
 "wgs84_lat": 51.520833013022035,
 "coordsyst": "G",
 "shortcuts": {"WMC": 65759, "ward": 8580, "council": 2504},
 "wgs84_lon": -0.14013153162152972,
 "postcode": "W1W 6UW",
 "easting": 529135,
 "areas": {"900000": {"parent_area": null,
                      "generation_high": 18,
                      "all_names": {},
                      "id": 900000, "codes": {},
                      "name": "House of Commons",
                      "country": "",
                      "type_name": "UK Parliament",
                      "generation_low": 1,
                      "country_name": "-", "type": "WMP"
                      },
          },
  "northing": 181852
}
```


JSON representation

Different representations but the same resource



W1W 6UW

- OSGB E/N: 529133, 181858
- WGS84 lat/lon: 51.5208323326, -0.14013206529

Get this data as JSON

Areas

Cities of London and Westminster
ID 65759, UK Parliament constituency (WMC), England

Greater London Authority
ID 2247, Greater London Authority (GLA), England

London
ID 11806, European region (EUR), England

HTML representation

Mapit REST documentation example

lookup by
postcode

URL:

/postcode/*[postcode]*

Returns:

Information on a particular postcode, including its centroid location in WGS84 latitude/longitude, and the areas that is contained within.

Optional query parameters:

generation, to return results from a previous generation.

Example:

If your postcode was *SW1A 1AA* you would load /postcode/SW1A1AA

By default the response is in JSON, if you want a human readable page then append '*.html*' to the url /postcode/SW1A1AA.html

From https://mapit.mysociety.org/docs/#api-by_postcode

Representations and their descriptions

- Requests for representations should include description of representation required(e.g, JSON, XML etc.)

- Responses should include description of representation - self-describing using MIME types in Content-Type response headers, e.g.,

  - `Content-Type: text/html`

  - `Content-Type: application/json`

REST principles

- Uniform interfaces: interfaces simple and consistent
- Representations - HTML, JSON, XML etc.
  - representations are *self-describing* - through `Content-Type` headers
- Statelessness: every request carries with it all data needed to service request - e.g, ***no cookies***
  - See https://nvoulgaris.com/designing-a-restful-shopping-cart/ and https://stackoverflow.com/questions/33786421/rest-shopping-cart - for how we might implement e.g. a shopping basket using REST
- Also: caching/layered architecture/client-server model

Elements of REST

- The key elements of REST are
  - nouns - “naming” words - on the web, nouns name resources
  - verbs - “doing” words - what you want to do with named resources

GET http://www.kitchen.com/utensils/cups/coffeecup

Get cup

VERBS

Request methods are verbs

NOUNS

mug

http://www.kitchen.com/utensil/cups/enamelcup

URLs are nouns for web resources

Verbs for the web

- Request methods are verbs for the web
- Describe the things we want to do with resources - semantics of request methods
- POST - Create
- PUT – Update. *(NOTE – PHP doesn't deal well with PUT)*
- DELETE - Delete
- GET - Fetch

GET

- Fetch things from server
- E.g., search results, list of products, product details
- Characteristics
  - Idempotent - always get the same result
  - Safe - no side-effects (data on server doesn’t change)
  - Cacheable - can remember result

- GET fetches representations
- REST APIs should allow you to choose representation
- E.g. (copy and paste these links into browser…)
- GET http://www.omdbapi.com/?s=expendables&r=xml&apikey=4bd83923
- GET http://www.omdbapi.com/?s=expendables&r=json&apikey=4bd83923

PUT

- Update something on the server
- Update quantity of item in shopping basket or avatar on Facebook profile
- No new data created
- Idempotent - always get same result

DELETE

- Delete some data
- Delete image from Facebook album, delete tweet, delete item from shopping basket
- Idempotent - if you try to delete something multiple times, data on server remains in same state

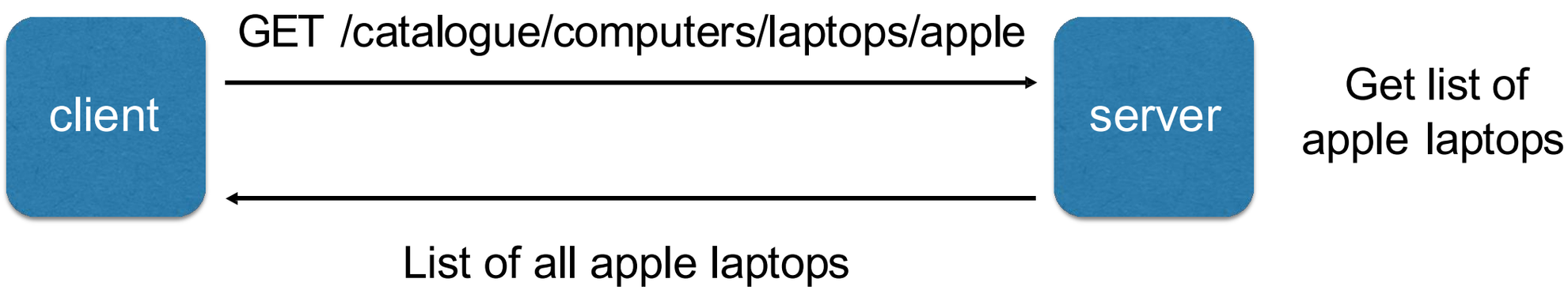
POST

- Create new data
- Register new user, add item to shopping basket, post new tweet, new Facebook status etc.
- Not idempotent, not safe, not cacheable…

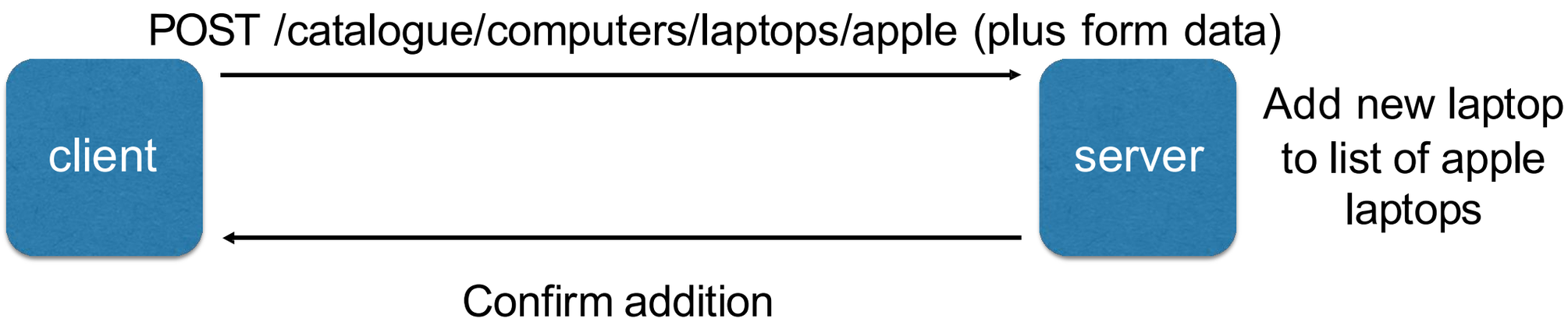
Verbs + Nouns

- What we want to do (create/fetch etc.) and what we want to do it to (which resource)
  - GET /catalogue/computers/laptops/apple
  - POST /catalogue/computers/laptops/apple
- These URLs reference the same thing but should result in different actions

GET /catalogue/computers/laptops/apple



POST /catalogue/computers/laptops/apple



Creating your own REST API

- Decide
  - what are your representations?
  - what are your actions and how do they map to request methods
  - what data formats will you use?
- CodeIgniter - use Phil Sturgeon's REST_Controller class https://github.com/chriskacerguis/codeigniter-restserver - see Blackboard link to CodeIgniter installation with REST Controller pre-installed

Case study

- Building a REST API for …. CUTE CATS!

How it will work

- GET index.php/restapi/cat/<NAME>
  - Will retrieve data about a particular cat called <NAME> (e.g, “fluffy”)
  - If no name given, then get ALL cats
- POST index.php/restapi/cat/<NAME>
  - Will create a new entry about a cat (together with form data)
- DELETE index.php/restapi/cat/<NAME>
  - Delete a cat
- PUT index.php/restapi/cat/<NAME>
  - Will update a cat (using form data)

URL structure

- Typical URL will be:
  - **`index.php/restapi/cat/fluffy`**
- **`restapi`** will be the controller
- **`cat`** - action in controller
- **`fluffy`** – URL segment data – retrieve in controller action

Getting started

- Download REST_Controller extension to CodeIgniter – see Blackboard link
- Note: example code in following slides uses an older version of REST_Controller – see Blackboard zip file for newer example
- Write controller etc.
- Test!

The CAT rest api

- http://localhost:8888/6cosc022w/cutecats/index.php/restapi

Add a cat

Name | Image URL | Add cat

Find a cat

fluffy

Find cat!

fluffy

Controller – part 1

```php
<?php defined('BASEPATH') OR exit('No direct script access allowed');

require APPPATH . '/libraries/RestController.php';
use chriskacerguis\RestServer\RestController;

class Restapi extends RestController {
    public function __construct()
    {
        parent::__construct();
        $this->load->helper('url');
        $this->load->model('catmodel');
    }

    public function index_get()
    {
        $this->load->helper('url');
        $this->load->view('cat_home'); // load initial interface
    }

```

This slide and next 2 contain code for controller class

Controller – part 2

```
20      /**
21       * Get cat data.  If name given, return data about a particular cat.  If
22       * no name given, return all cats.
23       * @return array containing cat data and count of number of cats returned.
24       */
25      public function cat_get()
26      {
27          // 2nd argument - what to return if segment is missing
28          $name = urldecode($this->uri->segment(3));
29          $cats = array();
30          if ($name == false) {
31              $cats = $this->catmodel->getcats();
32          }
33          else {
34              $name = $name;
35              $cat = $this->catmodel->getcat($name);
36              if ($cat !== false) {
37                  $cats = array($cat);
38              }
39          }
40          $data = array('cnt' => count($cats),'cats' => $cats);
41          log_message('DEBUG',print_r($data,true));
42          // print json_encode($data);
43          print json_encode($data);
44      }
45
```

Controller – part 3

```
45
46      function cat_post()
47      {
48          $name = $this->uri->segment(3,false);
49          if ($name === false) {
50              print json_encode(array('status' => 500,'msg' => 'bad data'));
51              return;
52          }
53          $img = $this->input->post('img');
54          $this->catmodel->set($name,$img);
55          print json_encode(array('status' => 200,'msg' => 'ok'));
56      }
57  }
58
```

Last part of the code for controller class – shows the method called when we request ’cat’ action with POST request method

REST : Good vs Bad

- Good
  - simple, clean interfaces
  - easy to build applications from other data sources
- Bad
  - Statelessness principle makes REST difficult when interaction spans multiple request/responses…
  - … or when you change devices and want to resume session (e.g, view shopping basket)

Testing REST APIs

- Postman https://www.getpostman.com
- Postman allows you to:
  - Create test suites for web apis
  - Run them automatically and log results

API Keys

- Many publicly-available REST APIs require API keys
- These effectively represent account management, allowing REST APIs owners to manage resource use
- Remember that REST APIs are stateless, so we can't use cookies to remember that someone is authenticated – instead we use API keys to know that a particular request came from a particular user
- Usually means registering for an account and getting allocated an API key, then using the key on requests

Example API key usage

- www.pexels.com - requires registration to get API key

- Ajax request:

```
var ajaxobj = $.ajax({
url : "https://api.pexels.com/v1/search?query=" + query + "&per_page=1",
dataType: 'json',
headers: {
    "Authorization": "SEs2dJxCryPGG9uJyyoDBA290RDl7yfrxQAPKOPwTTU7o16U3HBLY???"
},
method : 'GET'
}).done(function (data) {
        // see docs at https://www.pexels.com/api/documentation/#photos-search
        // for JSON structure
        // note - no error checking to see if there's actually an image there!
        var mediumimgurl = data.photos[0].src.medium;
        $('#photos').html("<img src='" + mediumimgurl + "'/>")
})
```

API Key example

- The API Key in the Pexel example is added as a header to the HTTP request

- Other sites require the API key to be included in the query string – check the documentation for the REST API

- E.g., OMDB (Open Movie Database) URLs - http://www.omdbapi.com/?apikey=[yourkey]&...

CodeIgniter and REST APIs

- Use the REST_Controller extension to CodeIgniter (see Blackboard website)
- Affects Controller classes only
- Define different versions of controller actions for different request methods
- Extension automatically maps different requests and request methods to right controller action

See also

- https://www.redhat.com/en/topics/api/what-is-a-rest-api
- https://aws.amazon.com/what-is/restful-api/
- https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm (Roy Fielding's PhD thesis on REST, Chapter 5)